



المدرسة العليا
للتكنولوجيا - الصويرة
L'ÉCOLE SUPÉRIEURE DE
TECHNOLOGIE – ESSAOUIRA

End-of-studies project dissertation

For Graduation as

Specialized Technician

In Business Intelligence and Data Science

Topic

Emotion and Gender Recognition Using Deep Learning Algorithms

Realized by:

OUGOUAMMOU Soukaina

BAMOULA Abdelghafour

Supervised By:

Pr. BENKIRANE Said

University Year 2021 / 2022

ACKNOWLEDGEMENT

It is a great pleasure to have the opportunity to extend our heartfelt gratitude to everyone who helped us throughout the course of this project. We are profoundly grateful to our supervisor Mr. Said BENKIRANE, for his expert guidance, continuous encouragement and ever willingness to spare time from his otherwise busy schedule for the project's progress reviews. His continuous inspiration has made us complete this project and achieve its target.

We would also like to express our deepest appreciation to all the faculty members for their support and encouragement, for the constant motivation, support and for providing us with a suitable working environment.

At last, our special thanks go to all the teachers of ESTE who directly and indirectly extended their hands in making this project work a success.

Summary

Abstract.....	7
Introduction.....	8
I- problem definition	10
II-Machine learning and deep learning basics	11
1- Machine learning	11
1.1 Definition:	11
1.2 types of machine learning	11
2- Deep learning :	12
2.1 – Deep learning functionality	12
3-CNN (Convolutional neural networks)	13
3.1 Definition :	13
3.2 CNN function	13
4-Scope of the project :	18
Conclusion :	18
I-methodology and technique	20
1-emotion recognition model	20
1.1model architecture.....	20
1.2 application model	20
1.3 Dataset preparation.....	21
1.3 CNN Implementation:	23
• Convolutional layers :	24
• Pooling :	24
• Flatten :	24
• Fully connected layer :	24
• Dropout :	25
• Output Layer	25
1.4 Evaluation Metrics :	25
2-GENDER RECOGNITION MODEL	29
2.1 architecture of the CNN model.....	29

2.2 dataset description	30
2.3 CNN Implementation	31
2.3 Performance accuracy	31
I-Tools used in our project	35
1-Programming languages :	35
2-Data pre-processing and model building tools	35
3-Implementation tools:	37
4-Other development tools:	37
II-use case diagram:	39
III-Project interfaces:	39
1-loading interface	40
2-Real time application interface	40
Conclusion :	42
3-classification by images interface	42

Figure list

Figure 1: Problem architecture.....	10
Figure 2: types of machine learning.....	11
Figure 3: CNN architecture.....	13
Figure 4: input images	14
Figure 5:convolutional layer functionality.....	14
Figure 6: filters operations.....	15
Figure 7: Max pooling	17
Figure 8: flatten and fully-connected CNN.....	17
Figure 9: Model architecture.....	20
Figure 10: Application design.....	20
Figure 11:Emotion distribution on the dataset.....	21
Figure 12:image to array	22
Figure 13:CNN architecture.....	23
Figure 14: emotion model confusion matrix.....	26
Figure 15: Plot for accuracy score and Loss distribution for train and test	28
Figure 16 :Gender detection model architecture	29
Figure 17: Gender data visualization	30
Figure 18: CNN architecture.....	31
Figure 19: emotion model confusion matrix.....	32
Figure 20: use case diagram.....	39
Figure 21: loading interface	40
Figure 22:Real time application interface	41
Figure 23: classify by image interface	42

Abbreviations

CNN	Convolution Neural Network
FER	Facial Expression Recognition
GPU	Graphics Processing Unit
ReLU	Rectified Linear Unit
ANN	Artificial Neural Network
MVC	Model View Controller

Abstract

A key step in the humanization of robotics is the ability to classify the emotion and gender of human operator, since it has place in wide range of application in human machine interaction, pattern recognition, image understanding, machine vision and other fields. However, it is a hard task to analyze human expression, due to the differences way how people express their sensation and feelings, also the effect of external factors like brightness, background and other factors. In addition, in our case of study, we applied some deep learning algorithms such as CNN to extract features from images in order to analyses and classify gender and facial expressions from both static images and videos to reveal information on one's emotional state. The operation mentioned bellow are used to get the best performance network and optimize the predictions. For the final exhibition of the model, we deployed it on a real time application that can instantaneously return the emotion of the user in addition of their gender.

Introduction

In the last decades, the detection of human emotions took place in various domains such as assisted living, health monitoring, domestic appliance control, crowd behavior tracking real time, and emotional security.

Human beings have recognition capabilities that are unparalleled in the modern computing Area. These are mainly due to the high degree of interconnectivity, adaptive, nature learning skills and generalization capabilities of the nervous system. The human brain has highly interconnected biological neurons, which, on some specific task can highly outperform. A human can accurately identify a face emotion, but for a computer it is a very hard task.

Therefore, the main idea is to engineer a model that can classify human emotion into categories and function like a nervous system. In order to optimize the performance we used CNN (convolutional neural network) which is a class of artificial neural network most commonly applied to analyze visual imagery.

Moreover, to achieve that goal, we choose to work with the FER-2013 for emotion recognition and UTK dataset for gender detection, because they are well-knowns datasets that contains a very large amount of images for different people. The task is to categorize each face based on the emotion shown in the facial expression with gender classification.

After training the model, we implement a real time classifier that can detect the faces and write it below.

Finally, in this paper we will explain our work following those steps:

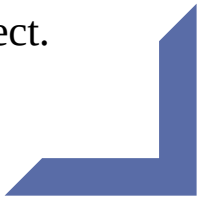
- Chapter I : in this chapter we will focus on the problem definition and basics of machine learning and deep learning algorithms, as well as the scope of the project
- Chapter II: for this chapter we will present the methodology and techniques that used for the model building.
- Chapter III: this chapter will represent the output of our model and the different tools that been used.

Chapter I

State Of Art



In this chapter, we will talk about the main problem of our project the solutions that we proposed in order to solve the problematic and the basics of deep learning and algorithm that we will used working in this project.



I- problem definition

Automatic recognition of facial expressions can be an important component of natural human-machine interfaces it may also be used in behavioral science and in clinical practice. In our case of study it will take place in psychiatrist's office especially in waiting room so before they start there session of therapy the psychiatrist will gets an idea of their patient state so they can adjust the optimized treatment for their patient.

In order to solve the problem of Facial Expression Recognition, we will adopt an idea that is based on processing the input images of human facial emotion to train the models on datasets, using deep learning technique called Convolutional Neural Networks (CNNs). to establish a classification model that combines feature extraction with classification .This projection constructs a system model to classify a given image of human facial emotion into one of the seven basic human emotion.

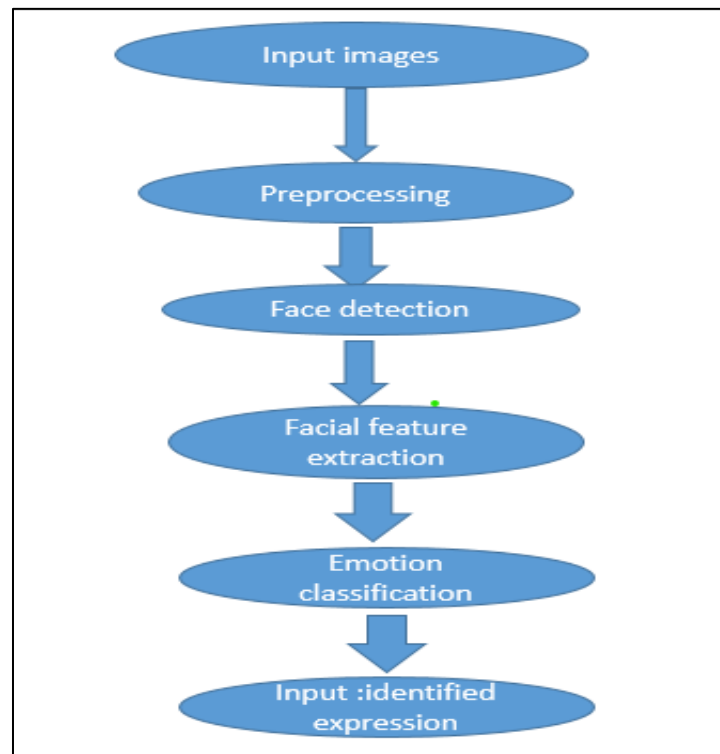


Figure 1: Problem architecture

II-Machine learning and deep learning basics

In this part we will explain the state of art of our project including the basics of deep learning and machine learning algorithms

1- Machine learning

1.1 Definition:

Machine learning is a branch of artificial intelligence (AI) and computer science which focuses on the use of data and algorithms to imitate the way that humans learn, gradually improving its accuracy.

1.2 types of machine learning

Machine learning can be divided into two parts of study :

- ✓ Supervised learning : is defined by its use of labeled datasets to train algorithms that to classify data or predict outcomes accurately. As input data is fed into the model, it adjusts its weights until the model has been fitted appropriately.
- ✓ Unsupervised learning: also known as unsupervised machine learning, uses machine learning algorithms to analyze and cluster unlabeled datasets. These algorithms discover hidden patterns or data groupings without the need for human intervention.
- ✓ Semi-supervised learning(Reinforcement): Semi-supervised learning offers a happy medium between supervised and unsupervised learning. During training, it uses a smaller labeled data set to guide classification and feature extraction from a larger, unlabeled data set.

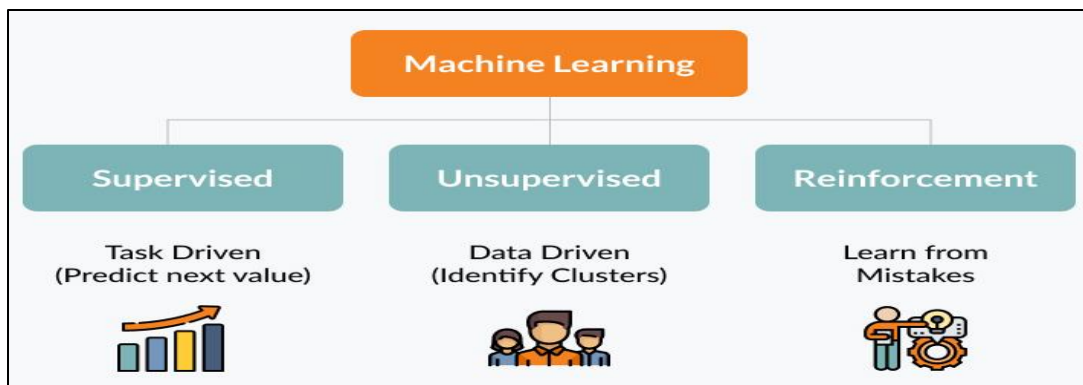


Figure 2: types of machine learning

2- Deep learning :

Deep learning is a subset of machine learning, which is essentially a neural network with three or more layers. These neural networks attempt to simulate the behavior of the human brain- albeit far from matching its ability-allowing it to “learn” from large amounts of data. While a neural network with a single layer can still make approximate predictions, additional hidden layers can help to optimize and refine for accuracy.

A neural network will always have:

- ✓ Input layer: It can be pixels of an image or a time series data
- ✓ Hidden layer: Commonly known as weights which are learned while the neural network is trained
- ✓ Output layer: Final layer mainly gives you a prediction of the input you fed into your network.

2.1 – Deep learning functionality

Deep learning neural networks, or artificial neural networks, attempts to mimic the human brain through a combination of data inputs, weights, and bias. These elements work together to accurately recognize, classify, and describe objects within the data.

However, deep learning algorithms are incredibly complex, and there are different types of neural networks to address specific problems or datasets. For example:

- ✓ Convolutional neural networks (CNNs), used primarily in computer vision and image classification applications, can detect features and patterns within an image, enabling tasks, like object detection or recognition. In 2015, a CNN bested a human in an object recognition challenge for the first time.
- ✓ Recurrent neural network (RNNs) are typically used in natural language and speech recognition applications as it leverages sequential or times series data.

3-CNN (Convolutional neural networks)

3.1 Definition :

A convolutional neural network is a special type of neural network that derives its work from biological processes present in the visual part of the human brain. The main component of CNNs are the convolutional layers and these layers are what make CNNs.

3.2 CNN function

Before we go to the working of CNN's let's cover the basics such as what is an image and how is it represented.

Technically, deep learning CNN models to train and test, each input image will pass it through a series of convolution layers with filters (Kernels), Pooling, fully connected layers (FC) and apply Softmax or sigmoid functions to classify an object with probabilistic values. The below figure is a complete flow of CNN to process an input image and classifies the objects based on values.

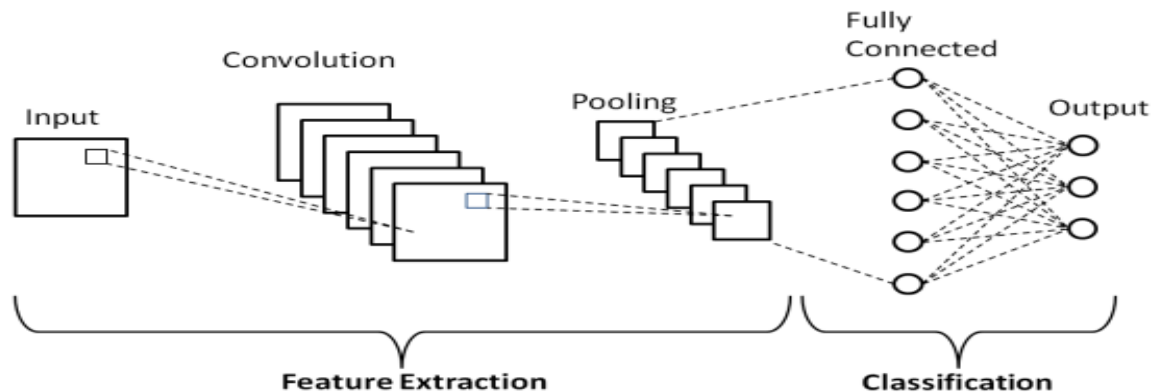


Figure 3: CNN architecture

- **Input images**

An RGB image is nothing but a matrix of pixel values having three planes whereas a grayscale image is the same but it has a single plane. The figure bellow explained more.

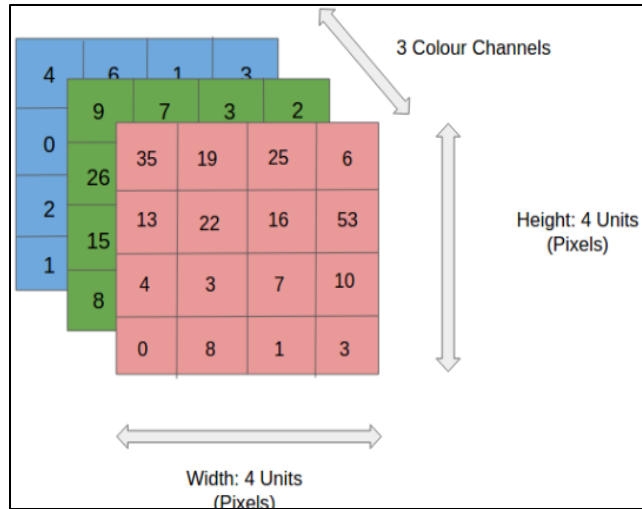


Figure 4: input images

CNN image classifications takes an input image, process it and classify it under certain categories . Computers sees an input image as array of pixels and it depends on the image resolution. Based on the image resolution, it will see $h \times w \times d$ (h = Height, w = Width, d = Dimension), An image of $6 \times 6 \times 3$ array of matrix of RGB (3 refers to RGB values) and an image of $4 \times 4 \times 1$ array of matrix of grayscale image.

- **Convolutional layer :**

Convolution is the first layer to extract features from an input image. Convolution preserves the relationship between pixels by learning image features using small squares of input data. It is a mathematical operation that takes two inputs such as image matrix and a filter or kernel.

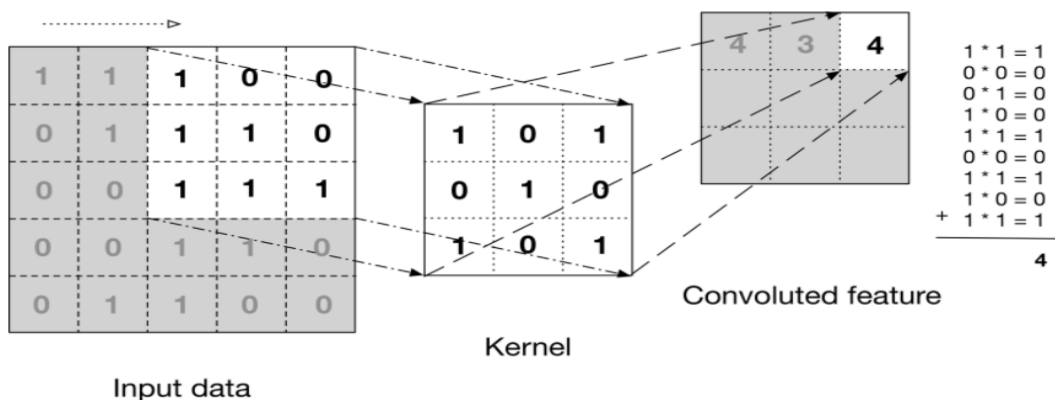


Figure 5:convolutional layer functionality

We take a filter/kernel(3×3 matrix) and apply it to the input image to get the convolved feature. This convolved feature is passed on to the next layer.

Convolutional neural networks are composed of multiple layers of artificial neurons.

The first layer usually extracts basic features such as horizontal or diagonal edges. This output is passed on to the next layer, which detects more complex features; such as corners or combinational edges. As we move deeper into the network, it can identify even more complex features; such as objects, faces, etc.

Convolution of an image with different filters can perform operations such as edge detection, blur and sharpen by applying filters. The below example shows various convolution image after applying different types of filters (Kernels).

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Figure 6: filters operations

- **Strides**

Stride is the number of pixels shifts over the input matrix. When the stride is 1 then we move the filters to 1 pixel at a time. When the stride is 2 then we move the filters to 2 pixels at a time and so on. The below figure shows convolution would work with a stride of 2.

- **Padding**

Sometimes filter does not fit perfectly fit the input image. We have two options:

- ✓ Pad the picture with zeros (zero-padding) so that it fits
- ✓ Drop the part of the image where the filter did not fit. It is called valid padding, which keeps only valid part of the image.

- **Mathematical functions**

- ✓ **Relu :**

ReLU –equation (1) – stands for Rectified Linear Unit for a non-linear operation. ReLU's purpose is to introduce non-linearity in our *ConvNet(Convolutional network)*. Since, the real world data would want our *ConvNet* to learn would be non-negative linear values.

$$ReLU(x) = \max(0, x) \quad (1)$$

- ✓ **Sigmoid :**

The main reason why we use sigmoid function –equation (2) – is because it exists between (0 to 1). Therefore, it is especially used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice

$$Sigmoid(x) = \frac{1}{1+e^{-x}} \quad (2)$$

- ✓ **Softmax:**

The softmax function –equation (3) – its domain is the set of all real numbers, and its range between 0 and 1. Hence, The sigmoid function is used as the activation function for multi-class classification problems where class membership is required on more than two class labels.

$$Softmax(x) = \frac{e^x}{\sum e^x} \quad (3)$$

- **Pooling Layer**

Pooling layers section would reduce the number of parameters when the images are too large. Spatial pooling also called subsampling or down sampling which reduces the dimensionality of each map but retains important information. Spatial pooling can be of different types:

- Max Pooling
- Average Pooling
- Sum Pooling

Max pooling takes the largest element from the rectified feature map. Taking the largest element could also take the average pooling. Sum of all elements in the feature map call as sum pooling.

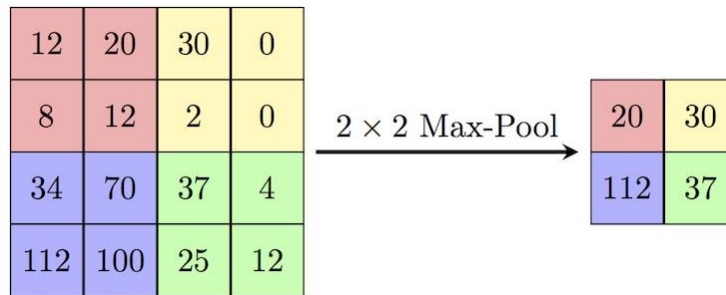


Figure 7: Max pooling

- **Fully connected layer and the flatten :**

the feature map matrix will be converted as vector (x1, x2, x3, ...). With the fully connected layers, we combined these features together to create a model. Finally, we have an activation function such as softmax or sigmoid to classify the outputs as a categories.

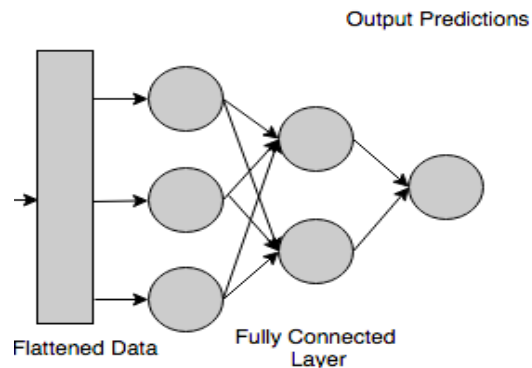


Figure 8: flatten and fully-connected CNN

4-Scope of the project :

In this project facial expression recognition and a gender detection system are implemented using convolution neural network. Facial images are classified into seven facial expression categories namely Anger, Disgust, Fear, Happy, Sad, Surprise and Neutral. Fer-2013 dataset is used to train and test the classifier.

An automatic Facial Expression Recognition and a gender detection system are build following those steps :

- Preprocessing phase :
 - Import fer-2013 dataset /import
 - Facial feature selection
 - Data visualization
 - Data augmentation
 - Data Normalization
- Model building :
 - Build a model using CNN
 - Train the model
 - Test the model
 - Analyze the performance metrics (which one is more optimized)
 - Solve overfitting problem
 - Model save
- Model deployment :
 - Model load
 - Real time application
 - Build a graphic interface to exhibit the final result

Conclusion :

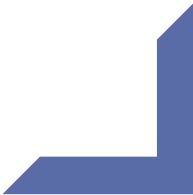
In this chapter we have talked about the main problem of our project the solutions that we proposed in order to solve the problematic and the basics of deep learning and algorithm that we will used working in this project.

Chapter II

Model Building



In this chapter, we will see the methodology that we followed and all the techniques that we used to build our model.



I-methodology and technique

1-emotion recognition model

1.1model architecture

Our model will be build following those specific steps as the schema shows bellow ,our model will firstly take a grayscale images from the train dataset, apply some preprocessing steps like data augmentation and data normalization to prepare the data in the most meaningful way for the subsequent detailed analysis. After we get the cleaned data, we input it into a CNN architecture to extract the feature, and then start a neural network model that will give us an output that contain the probabilities of each class.

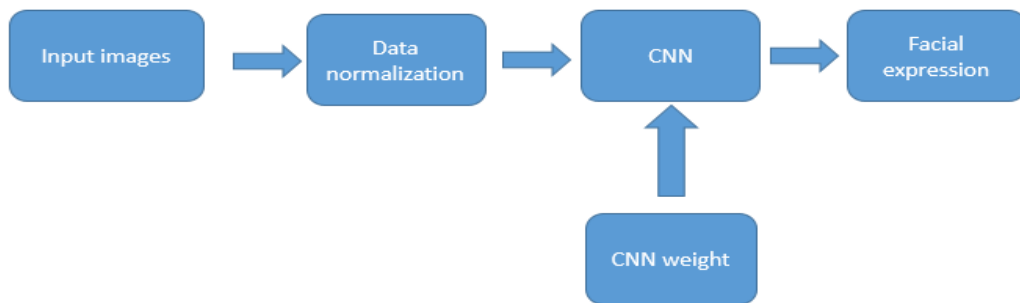


Figure 9: Model architecture

1.2 application model

The figure bellow shows the final model of our application, so as it shows firstly we will get input images from a testing dataset it can be images or real time ,and then our final CNN model will test the inputs and gives us the prediction that is classify in one of the 7th emotions .

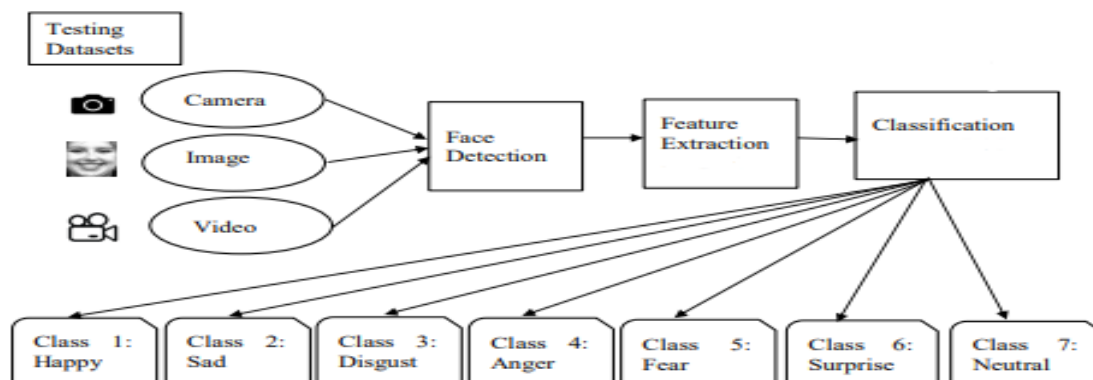


Figure 10: Application design

1.3 Dataset preparation

1.3.1 dataset description

The FER 2013 dataset is well known and was used in the Kaggle competition. The data consists of 48x48 pixel grayscale images of faces. The faces have been automatically registered so that the face is more or less centered and occupies about the same amount of space in each image. The data must be prepared for input to the CNN because there are some issues with this dataset as discussed below.

Working with the FER 2013 dataset we have faced a problem of imbalanced data ;

- **Imbalance dataset :**

Imbalance is when one class has many more images than another class. This results in the model being biased towards one class. For example, if there are 2000 images for the happy expression and 500 images for the fear expression, then the model will be biased towards the happy expression.

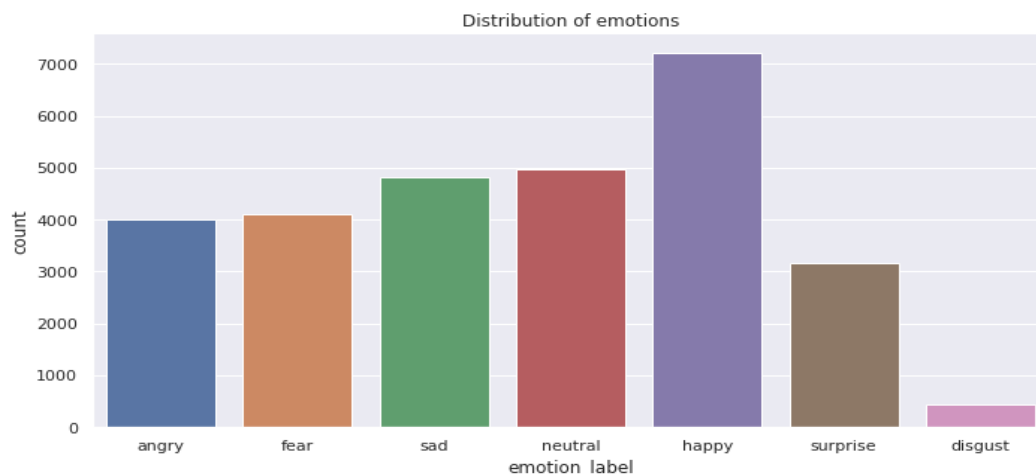


Figure 11: Emotion distribution on the dataset

As we see, the number of images of disgusting people is too small compared with the other categories.

1.3.3 Data preprocessing

In the preprocessing part we have applied data augmentation , data normalization and transforming pixels into numpy array:

➤ **Data augmentation :**

is done to avoid this problem. Data augmentation increases the amount of data using techniques like cropping, padding, and horizontal flipping.

- ✓ For this task we used a keras method that called ImageDataGenerator which is a technique of applying different transformations class provides a quick and easy way

to augment your images. It provides a host of different augmentation techniques like standardization, rotation, shifts, flips, brightness change to original images which results in multiple transformed copies of the same image.

Another task that shows during our preprocessing is that we added to each emotion class an index for that we used a map method to associate each class to an index.

➤ **Image to array**

- ✓ The data set contain grayscale images of faces so the next task is to convert those images into a nd array :

An image is represented by values (numbers) that correspond to the pixel intensities. The array module in NumPy (nd.array) is used to convert an image into an array and obtain the image attributes. Figure 2.2 shows an image in the sad class from the FER 2013 dataset converted into a NumPy array. Figure 2.3 shows the attributes of this image which are 2304 pixels, 2 dimensions and size 48×48 pixels.

In order to adjust the shape of the images in our dataset we added a function that reshape the pixels in 2 dimension and size 48x48 pixels.

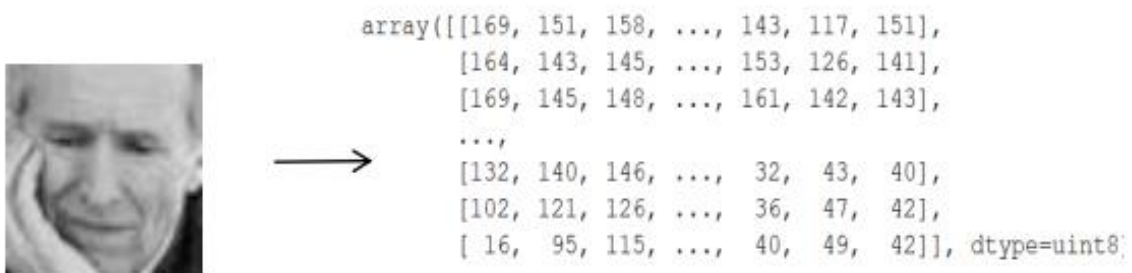


Figure 12:image to array

➤ **Data normalization :**

To avoid undesirable influence problem of high weights, we normalize continuous features values. The Equation is used to find the new values for the normalization phase we used the max min technique.

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

1.3 CNN Implementation:

ML models can be built and trained easily using a high level Application Programming Interface (API) like Keras. In this report, a sequential CNN model is developed using Tensorflow with the Keras API since it allows a model to be built layer by layer. CNNs layers are known to imitate how the human brain works when analyzing visuals.

For the CNN implementation we will follow the following architecture

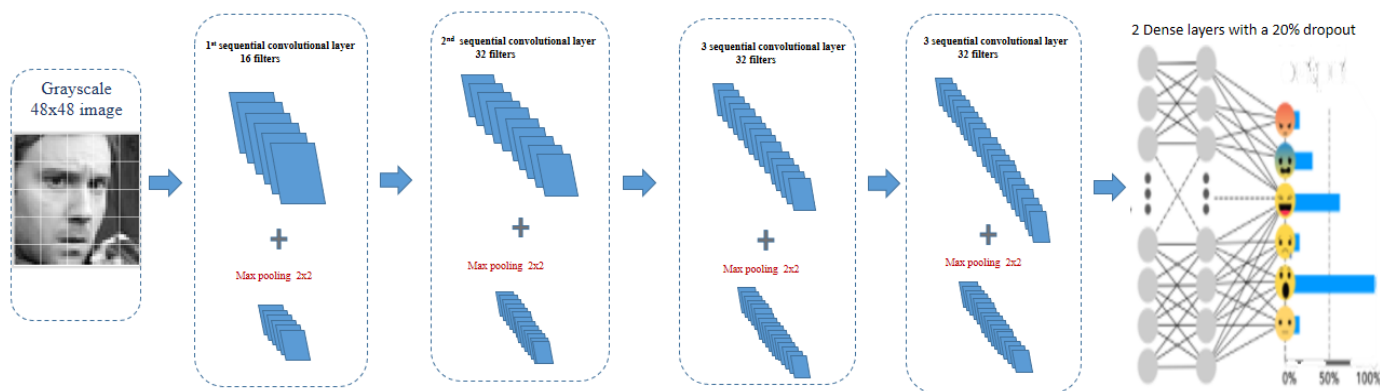


Figure 13:CNN architecture

- **Input layer:**

- ✓ **For the training phase:**

We will be using the FER2013 dataset as we explained before this dataset contains 48x48 pixels grayscale images of faces.

The first phase of the model has an input layer for an image of size 48×48 (height and width in pixels) and convolution is performed on this input.

- ✓ **For the test phase:**

OpenCV, a computer vision library, for face detection in the image, will apply the input layer for real time application. The haar-cascade_frontalface_default.xml in OpenCV contains pre-trained filters and uses Adaboost to quickly find and crop the face.

The cropped face is then converted from RGB image into grayscale image, and resized to 48-by-48 pixels with cv2.resize. This step greatly reduces the dimensions compared to

the original RGB format with three color dimensions (3, 48, 48). The pipeline ensures every image can be fed into the input layer as a (1, 48, 48) numpy array.

- ***Convolutional layers :***

The numpy array will be added into a convolutional2D, in our approach we have worked with 4 convolutional layers, the first convolutional layer we used 16 filters with a size of 3x3, The second convolutional layer had 32 3x3 filters, the third one had 64 3x3 filters and the last one had 128 3x3 filters. In all the convolutional layers, we have a stride of size 1, batch normalization, dropout with 25%, max-pooling with a pool size of 2x2 and ReLU as the activation function.

- ***Pooling :***

Is a dimension reduction technique usually applied after one or several convolutional layers. It is an important step when building CNNs as adding more convolutional layers can greatly affect computational time. we used a popular pooling method called MaxPooling2D that uses (2, 2) windows across the feature map only keeping the maximum pixel value. The pooled pixels form an image with dimensions reduced by 4.

- ***Flatten :***

At the last hidden layer we applied the flatten method to convert all the previous matrix into a one dimension vector to be the next inputs for our neural network.

- ***Fully connected layer :***

Is inspired by the way neurons transmit signals through the brain. It takes a large number of input features and transform features through layers connected with trainable weights.

These weights are trained by forward propagation of training data then backward propagation of its errors. Back propagation starts from evaluating the difference between prediction and true value, and back calculates the weight adjustment needed to every layer before. We can control the training speed and the complexity of the architecture by tuning the hyper-parameters, such as learning rate and network density. As we feed in more data, the network is able to gradually make adjustments until errors are minimized.

- **Dropout :**

The model also becomes increasingly prone to overfitting the training data. One method to prevent overfitting and generalize on unseen data is to apply dropout. Dropout randomly selects a portion (usually less than 50%) in our case it is 25% of nodes to set their weights to zero during training. This method can effectively control the model's sensitivity to noise during training while maintaining the necessary complexity of the architecture.

- **Output Layer**

Instead of using sigmoid activation function, we used softmax at the output layer. This output presents itself as a probability for each emotion class.

Therefore, the model is able to show the detail probability composition of the emotions in the face.

1.4 Evaluation Metrics :

Evaluation metrics are used to measure the quality of the model. There are many different types of evaluation metrics available to test a model, include classification accuracy, confusion matrix, and others.

1.4.1 Confusion matrix

Gives us a matrix as output and describes the complete performance of the model. There are other evaluation metrics that can be used like precision, recall, f-score. Evaluation metrics involves using a combination of these individual evaluation metrics to test a model.

The figure below shows us the confusion matrix given after testing our model using the test dataset.

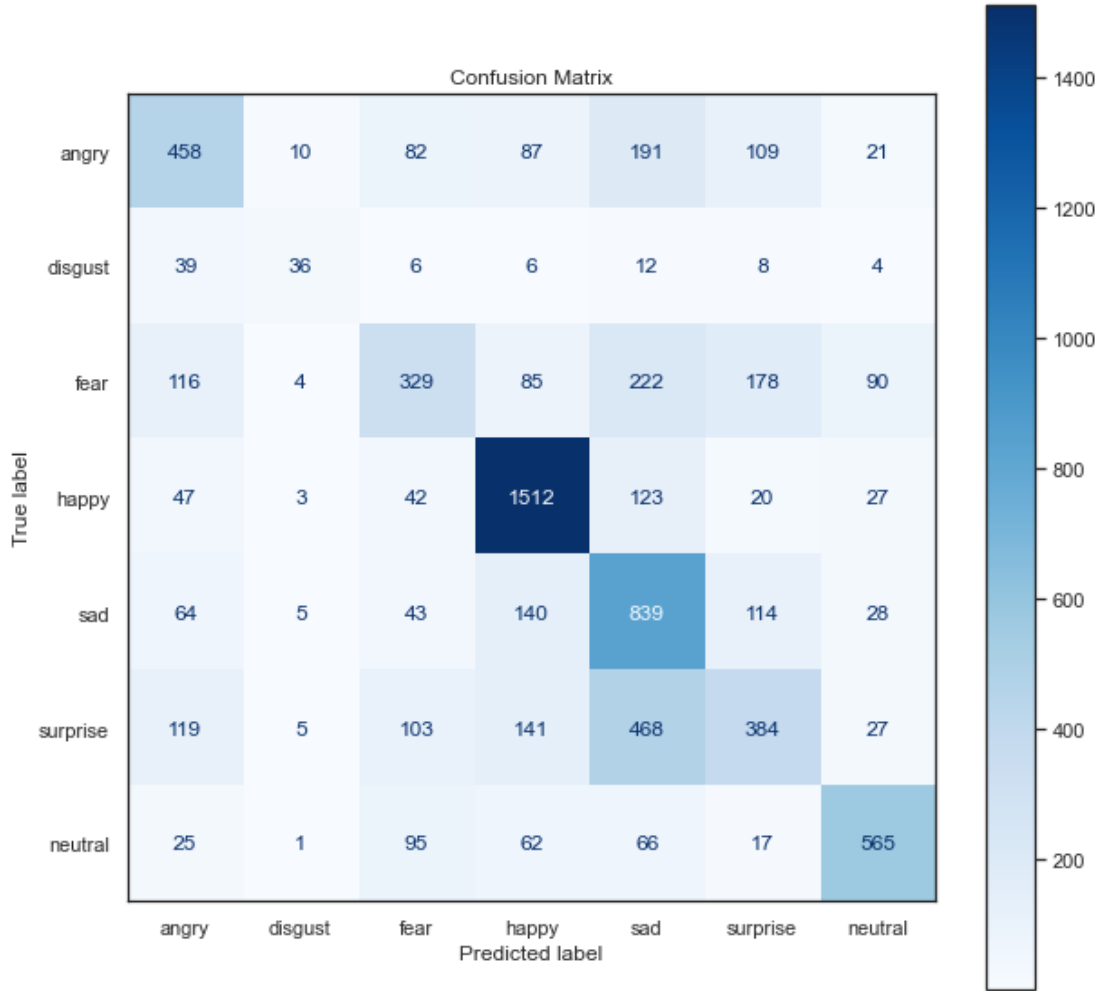


Figure 14: emotion model confusion matrix

- ✓ **Accuracy** is often the most used metric representing the percentage of correctly predicted observations, either true or false. To calculate the accuracy of a model performance, the following equation can be used:

$$accuracy = \frac{TP + TN}{TP + FN + FP + TN} \quad (2)$$

- ✓ The **recall** -equation (3)- represents the total number of positive classifications out of the true class. In our case, it represents the number of articles predicted as true out of the total number of true articles.

$$recall = \frac{TP}{TP + FN} \quad (3)$$

- ✓ The **precision** score -equation (4)- represents the ratio of true positives to all events predicted as true. In our case, precision shows the number of articles that are marked as true out of all the positively predicted (true) articles:

$$precision = \frac{TP}{TP + FP} \quad (4)$$

- ✓ **F1-score** -equation (5)- represents the trade-off between precision and recall. It calculates the harmonic mean between each of the two. Thus, it takes both the false positive and the false negative observations into account.

$$f1 - score = 2 \times \frac{precision \times recall}{precision + recall} \quad (5)$$

Our model is given an accuracy of 63% of training set and 57% for the test set as the table below mentioned

Table 1: Accuracy score and Loss for train and test

Emotion model	Accuracy	Loss
Training	63,55 %	0,96
Test	57,43 %	1,12

1.4.2 Over-fitting problem:

In the phase of training our model we have faced a problem called over-fitting which occurs when a statistical model fits exactly against its training data. When this happens, the algorithm unfortunately cannot perform accurately against unseen data. and for defeating this problem we tried to :

- ✓ Reduce the network's capacity by removing layers or reducing the number of elements in the hidden layers.
- ✓ Apply regularization (Batch Normalization) for each NN Layer (Dense) , which comes down to adding a cost to the loss function for large weights.

- ✓ Use Dropout layers, which will randomly remove certain features by setting them to zero.

the figure below show us the learning curve for the model before and after solving over-fitting problem.

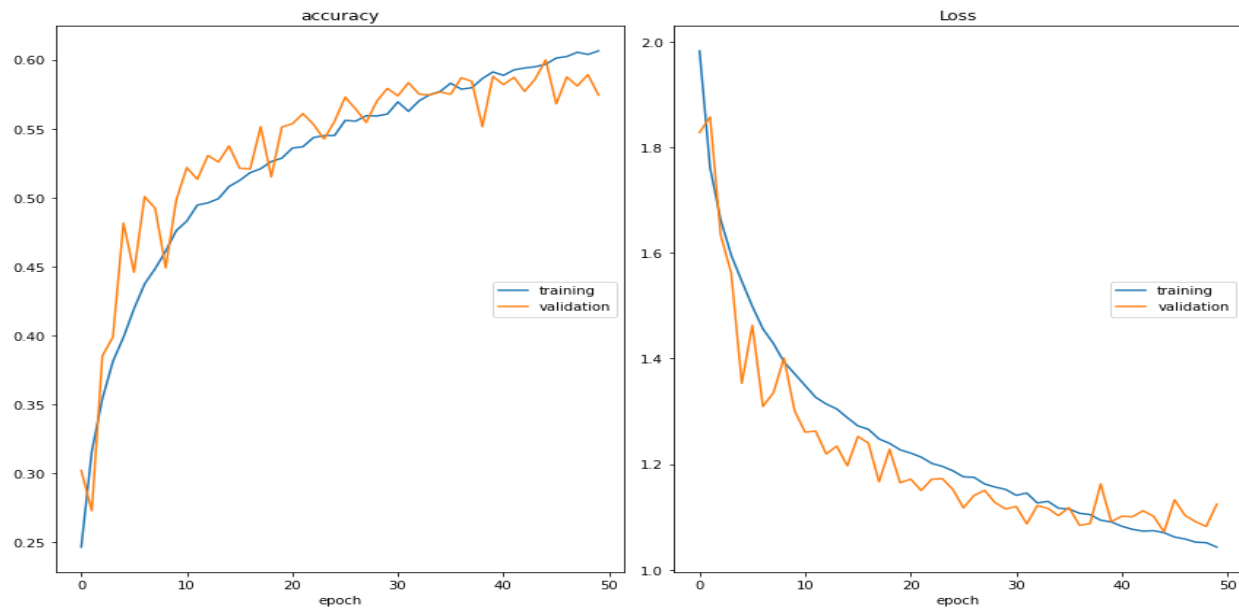


Figure 15: Plot for accuracy score and Loss distribution for train and test

1.4.3 Classification report

A classification report is a performance evaluation metric in machine learning. It is used to show the precision, recall, F1-score, and support of your trained classification model. As the table below present. Support is the number of actual occurrences of the class in the specified dataset. Imbalanced support in the training data may indicate structural weaknesses in the reported scores of the classifier and could indicate the need for stratified sampling or rebalancing.

Table 2: emotion gender classification report

	precision	recall	F1-score	support
angry	53 %	48 %	50 %	958
disgust	56 %	32 %	41 %	111
fear	47 %	32 %	38 %	1024
happy	74 %	85 %	79 %	1774
sad	44 %	68 %	53 %	1233
surprised	46 %	31 %	37 %	1247
neutral	74 %	68 %	71 %	831
accuracy			57 %	7178
macro age	56 %	52 %	53 %	7178
weighted age	57 %	57 %	56 %	7178

2-GENDER RECOGNITION MODEL

2.1 architecture of the CNN model

For the gender detection model it will be build following the steps shown bellow ,our model will firstly take images from the train dataset, apply some preprocessing like data normalization to prepare the data in the most meaningful way for the subsequent detailed analysis. After we get the cleaned data, we input it into a CNN architecture to extract the feature, and then start a neural network model that will give us an output that contain the probabilities of each class.

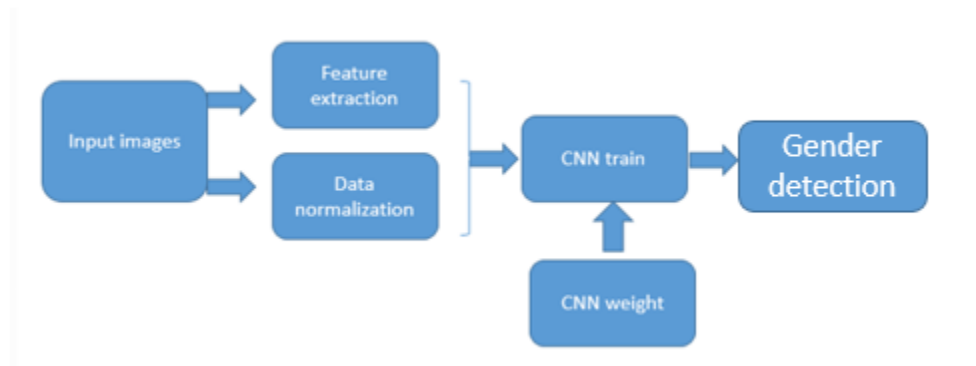


Figure 16 :Gender detection model architecture

2.2 dataset description

The UTKFace dataset is a large-scale face dataset with long age span (range from 0 to 116 years old). The dataset consists of over 20,000 face images with annotations of age, gender, and ethnicity. The images cover large variation in pose, facial expression, illumination, occlusion, resolution, etc.

We trained the model in age_gender.csv dataset. The data consists images of faces. The faces have been automatically registered so that the face is more or less centered and occupies about the same amount of space in each image. Except that the images are already transformed into pixels and loaded in a CSV file.

CSV file containing five columns age, ethnicity, gender, img_name, and pixels. We converted the column-named pixels into an array by using the NumPy library and reshaping them into dimensions 48, 48 by using the lambda function. We also converted the values in the float by the same lambda function.

After that we normalized the dataset divided the values further by 255.

The task is to categorize each face based on the gender and the appropriate age depending on the class where it can be put on , The training set consists of 17778 examples and the public test set consists of 5927 examples.

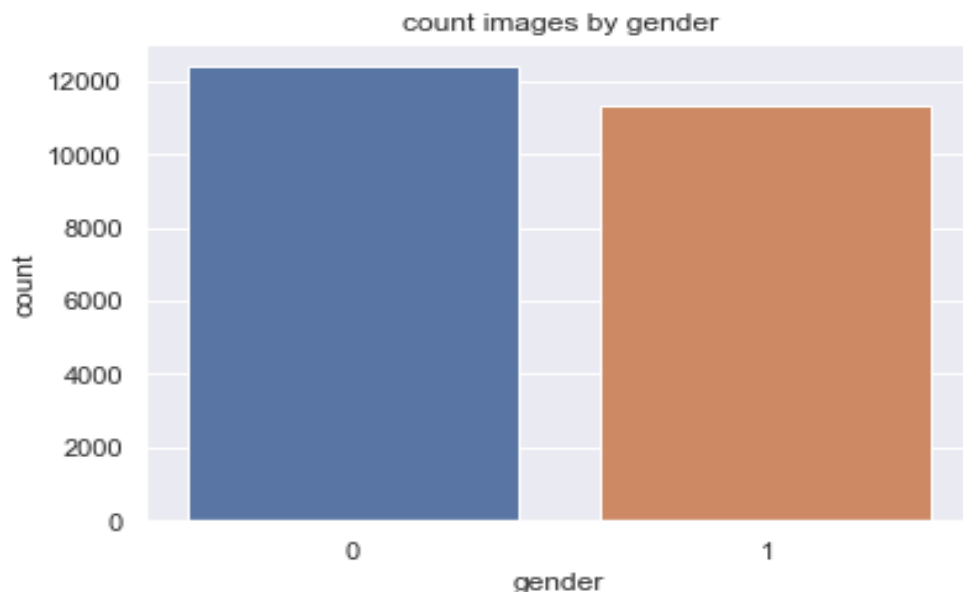


Figure 17: Gender data visualization

2.3 CNN Implementation

CNN architecture for age and gender detection as mentioned above was implemented in Python. Along with Python programming language, Numpy, tensorflow ,keras libraries were used.

For the convolutional layers architecture we have applied four convolutional layer , four max pooling with a pool size of (2,2) , after we get the last pixels matrix we convert it into a vector of 1 dimension that's what we called the flatten and at the end we applied two fully connected to reduce the number of connections . in addition , we have the output layer ,Output from the 4th hidden layer is connected to output layer having 2 classes for . Using Softmax activation function for sex, output is obtained using the probabilities for each of the classes. The class with the highest probability is the predicted class.

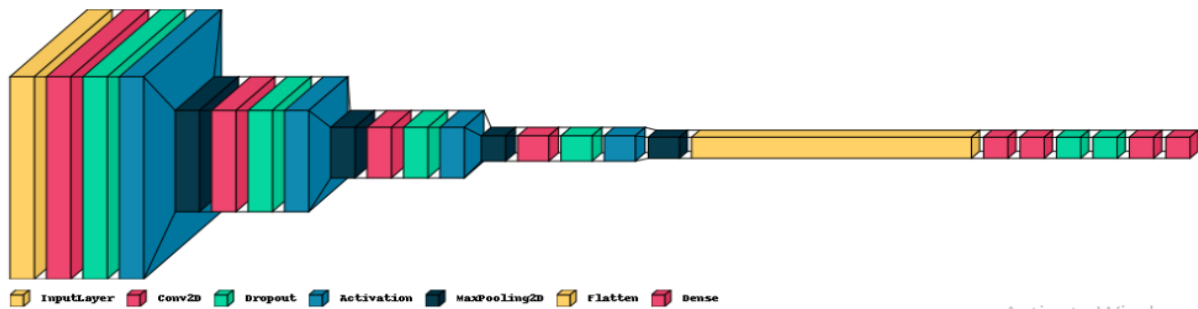


Figure 18: CNN architecture

2.3 Performance accuracy

2.3.1 accuracy and loss:

In this part we test the gender model the same metrics used in the previous model as the table 4 show.

Table 4: gender model accuracy and loss for training and test

gender model	Accuracy	Loss
Training	99,25 %	0.022
Test	89,34%	0.394

2.3.2 Confusion matrix

The confusion matrix below represents the number of correct and incorrect predictions are summarized with count values and broken down by each class.

Table 4: confusion matrix

	Female	Male
Female	TP	FN
Male	FP	TN

And this figure is a numeric representation for the values

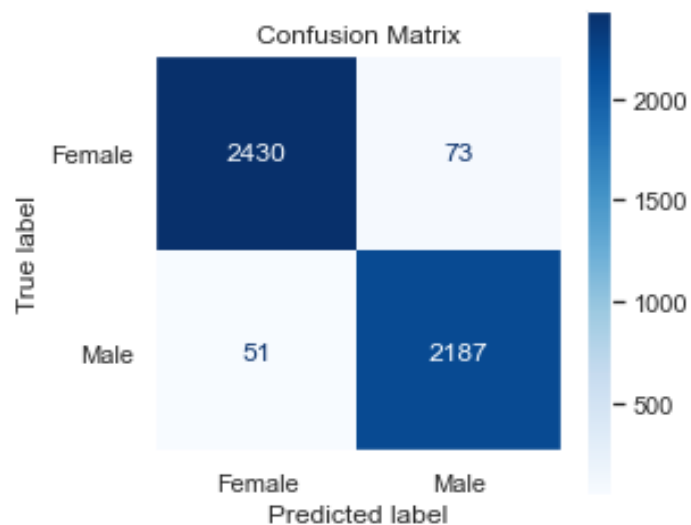


Figure 19: emotion model confusion matrix

2.3.3 Classification report

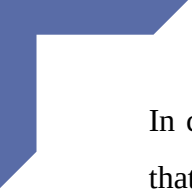
The classification report -table 6- shows a representation of the main classification metrics on a per-class basis. And gives a summary of prediction results on a classification problem

Table 3: Classification Report

	precision	recall	F1-score	support
Female	0.98 %	97 %	98 %	2503
Male	97 %	98 %	97 %	2238
accuracy			97 %	4741
macro age	97 %	97 %	97 %	4741
weighted age	97 %	97 %	97 %	4741

Chapter III

Implementation



In chapter will represent the output of our model and the different tools that been used. And the different tools that been used.



I-Tools used in our project

1-Programming languages :

We use for our work the Python 3 programming language for building the model also for implement it because it is a well-known language for machine learning and deep learning. And its provide many height level frameworks and libraries for those approaches. In addition to the big community and good documentation.



Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small- and large-scale projects.

2-Data pre-processing and model building tools

For the data pre-processing like loading data, data visualization, feature selection... and also for building our CNN model, train it, evaluate it and make prediction we use some frameworks and libraries for Python (3.9) languages such as numpy, pandas, matplotlib, seaborn, TensorFlow, keras.



TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks.



Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation.



NumPy stands for Numerical Python, it was created in 2005 by Travis Oliphant. It is an open source project and can be used for working with arrays and it also functions in other domains like linear algebra, Fourier transform and matrices.



Pandas is a software library written for the Python programming language for data manipulation and analysis. In particular, it offers data structures and operations for manipulating numerical tables and time series. It is free software released under the three-clause BSD license.



Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits.



Seaborn is a library for making statistical graphics in Python. It builds on top of matplotlib and integrates closely with pandas data structures.

3-Implementation tools:

For implement our model we use PyQt5 for develop a desktop application that can make a real time prediction or classification by image for this part we combine it with OpenCV.

The PyQt5 user interfaces based on MVC (Model-View-Controller) which is a pattern used to implement user interfaces, data, and controlling logic. It emphasizes a separation between the software's business logic and display.



OpenCV is a great tool for image processing and performing computer vision tasks. It is an open-source library that can be used to perform tasks like face detection, objection tracking, landmark detection, and much more.



PyQt is a Python binding of the cross-platform GUI toolkit Qt, implemented as a Python plug-in. PyQt is free software developed by the British firm Riverbank Computing. It gives programmers the freedom to create GUIs of their choice while also providing a lot of good pre-built designs. PyQT gives you widgets to create complex GUIs.

4-Other development tools:

There are many other packages and editors (applications) used for the implementation such as Anaconda packages for python which provides different tools needed for a data scientist as well as some editors like Jupyter and Spyder. also google collaboration which provide an environment for training deep learning models with TensorFlow and others.

in addition to Qt-Designer forms for creating Qt-templates and using them with PyQt.



Anaconda is a distribution of the Python and R programming languages for scientific computing, that aims to simplify package management and deployment. The

distribution includes data-science packages suitable for Windows, Linux, and macOS.



Project Jupyter is a project and community whose goal is to "develop open-source software, open-standards, and services for interactive computing across dozens of programming languages". It was spun off from IPython in 2014 by Fernando Pérez and Brian Granger.



Colab is the commonly used abbreviation of the New York City artists' group Collaborative Projects, which was formed after a series of open meetings between artists of various disciplines.



Spyder is an open-source cross-platform integrated development environment (IDE) for scientific programming in the Python language.



Visual Studio Code is a source-code editor made by Microsoft for Windows, Linux and macOS. Features include support for debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git.



Git is an open-source, version control tool created in 2005 by developers working on the Linux operating system; GitHub is a company founded in 2008 that makes tools which integrate with git. You do not need GitHub to use git, but you cannot use GitHub without using git.

II-use case diagram:

As the figure below represent When the user aces in the application he can chose a real time classification or by image. in real time he can start capture camera video and start make predictions. And in the classification by image the user should use an image then the application will make predictions and show the result.

Visual Paradigm Online Free Edition

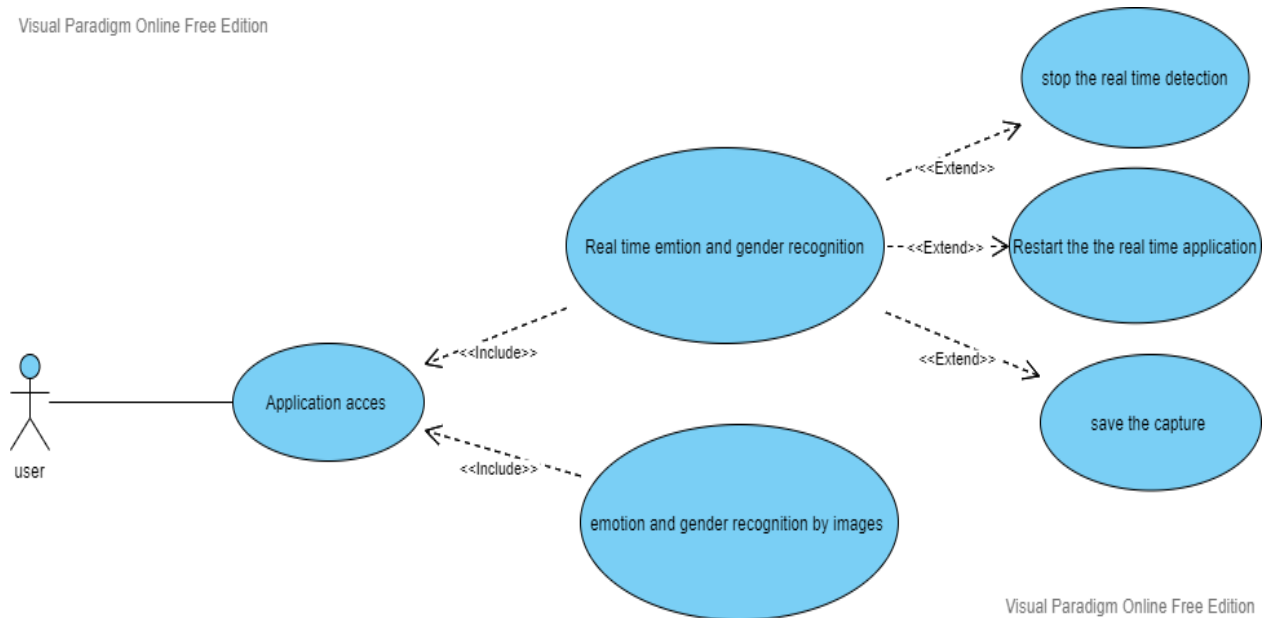


Figure 20: use case diagram

III-Project interfaces:

The following interfaces were built by Qt5 designer which is a tool for quickly building graphical user interfaces with widgets from the *Qt GUI framework*. It gives you a simple drag-and-drop interface for laying out components such as buttons, text fields, combo boxes and more.

1-loading interface

This is the first interface that shows after we compile the model

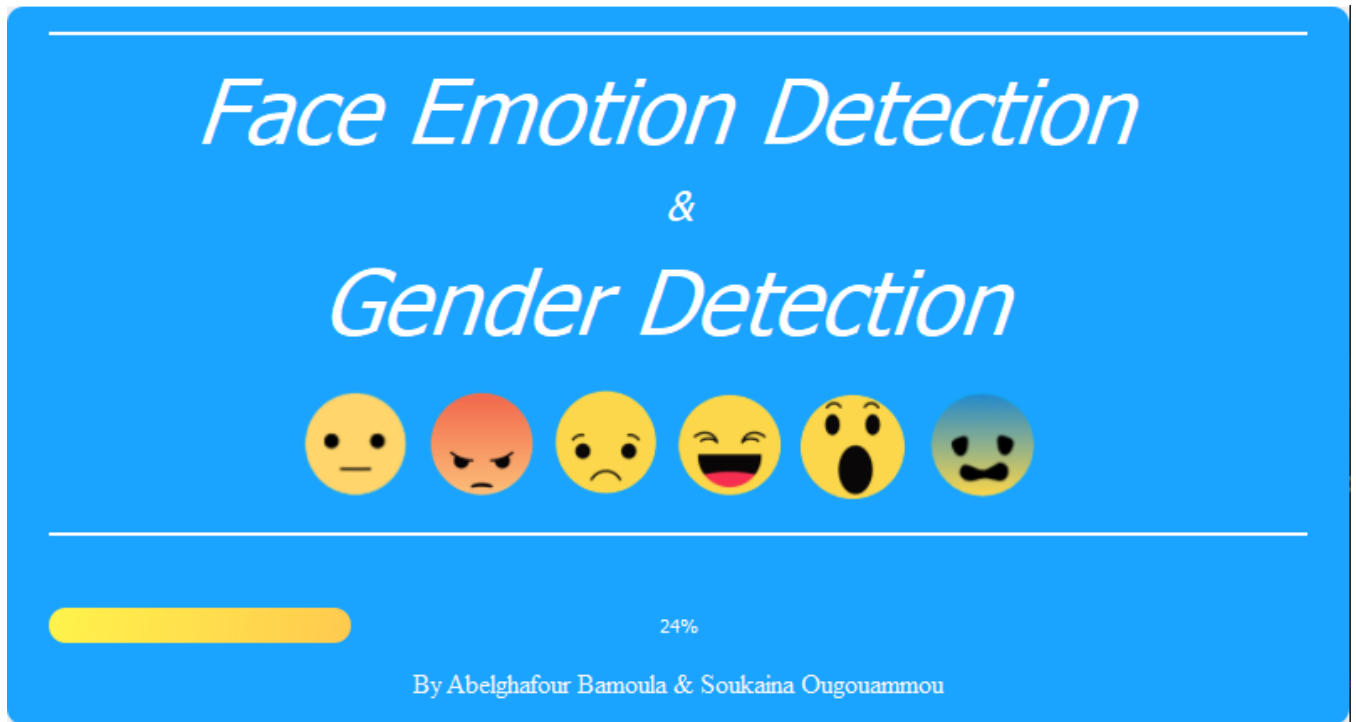


Figure 21: loading interface

This interface contain the title of our project also as a progress bar to show the progress of the loading until it shows the second interface

2-Real time application interface

The second interface is for the real time emotion recognition , for that we applied open cv for the camera capture and the detection of the human facial expression also as the gender detection



Figure 22:Real time application interface

- **The stop button:** capture button is used to stop the real time application in order to get a capture of the emotion state of the user .
- **The capture button :** this button is used to restart the real time application after we stop it with the stop button
- **The save button:** this button is used to save the prediction of the user
- **The By Images button :** this button is used to import images from our desktop files and predict the emotion state of the image
- **Exit button :** this button is used to return to the main page of our application .

3-classification by images interface

The third interface allow the user to import an image by click on 'load image'. And when he clicks on make prediction button the image will shown on the left with the prediction and number of faces detected. also, in the right there are two plots of predictions for each model.

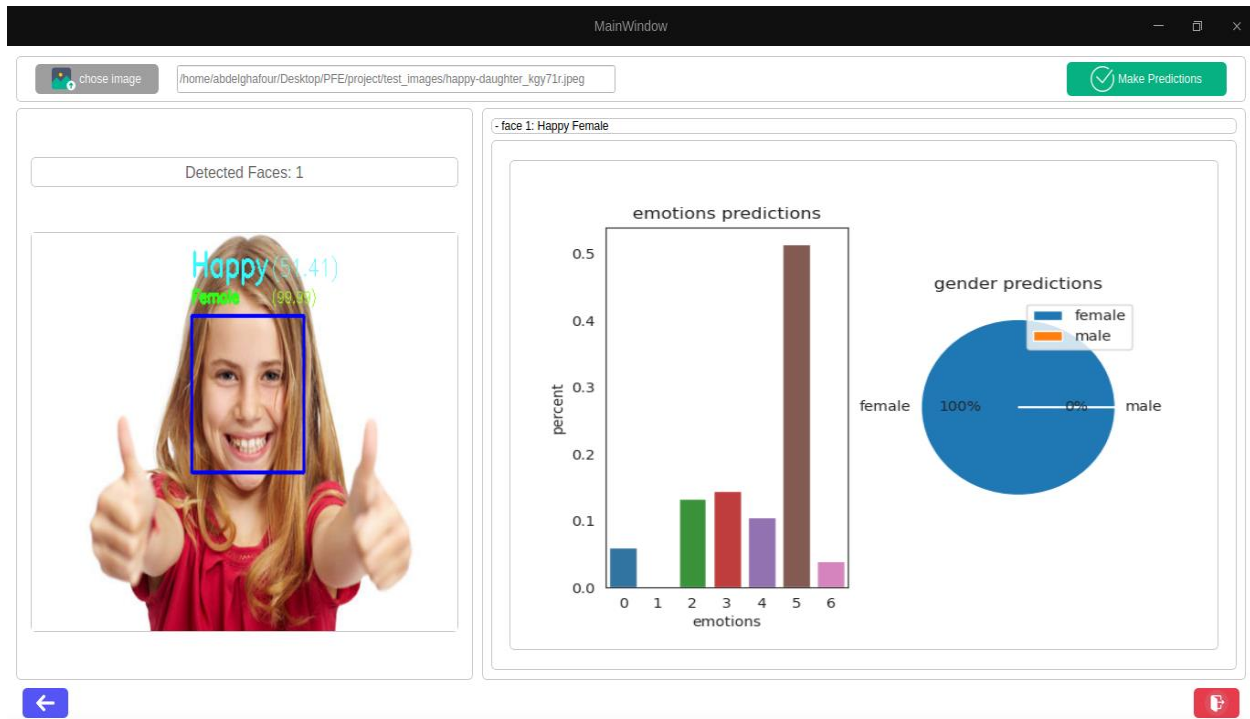


Figure 23: classify by image interface

Conclusion :

In this chapter, we have seen the languages and the IDE that we used during the implementation of our project also as the output interfaces of our project

Conclusion

The human face is an important organ of an individual's body and it especially plays an important role in extraction of an individual's behavior and emotional state. Manually detecting the emotions of facial features is a very tedious, time consuming, labor intensive and upheld task.

In order to realize this project, we went through several stages. This report reflects the effort made during the internship. Firstly, the work consisted of an initiation with the basics of machine learning and deep learning to discover the working environment, where we carried out a training that was rich in new notions of all the basics of our project. As well as some knowledge on the preprocessing of the data, which is a major part during the realization of our project, since it is based on the preparation of the data; to have more understandable dataset. In second place comes the implementation phase where we proposed an approach for recognizing the category of facial expressions using the CNN algorithm and for the final task we had built some python interfaces to show the output of our model.

This project was a great opportunity to try new things includes the group work that gives us so many benefits like; Break complex tasks into parts and steps, Plan and manage time Refine understanding through discussion, explanation, Give and receive feedback on performance and Develop stronger communication skills.

This knowledge, which we intend to use wisely, reinforces our choices of career and fuels our ambition to continue on this path, being certain that the donate science cap with the knowledge acquired at school; will serve us and that it will be a real plus compared to the others.

References:

- [1] M. Cabanac, "What is emotion?," Behavioural processes, vol. 60, pp. 69-83, 2002.
- [2] M.Sambhare, FER-2013 database, version 1, available online: <https://www.kaggle.com/msambhare/fer2013/metadata> (2013).
- [3] A Comprehensive Guide to Convolutional Neural Networks — the ELI5: way <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>
- [4] A. Amini, A. Soleimany, MIT deep learning open access course 6.S191, available online: <http://introtodeeplearning.com/> (2020).
- [5] S. Shah, A comprehensive guide to convolutional neural networks, available online: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-theeli5-way-3bd2b1164a53>, (2018)
- [6] S. S. Farfade, M. Saberian, L.-J. Li, Multi-view face detection using deep convolutional neural networks, arXiv preprint arXiv:1502.02766 (2015).
- [7] G. Guo, G. Mu, Joint estimation of age, gender and ethnicity: Cca vs. pls, in: Automatic Face and Gesture Recognition (FG), 2013 10th IEEE International Conference and Workshops on, IEEE, 2013, pp. 1–6.
- [8]] B. A. Golomb, D. T. Lawrence, T. J. Sejnowski, Sexnet: A neural network identifies sex from human faces., in: NIPS, volume 1, 1990, p. 2.

FIN